



Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Gelişmiş Konular ve Optimizasyon

PyGame ile Oyun Geliştirme – Hafta 13

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

- 1 Performans Optimizasyonu
- 2 Gelişmiş Çarpışma
- 3 Temel AI Teknikleri
- 4 Oyun Dağıtımı

Performans Optimizasyonu

- Hedef 60 FPS $\Rightarrow$ her kare için **16.67 ms**
- 30 FPS için 33.3 ms (acil eşik)
- Olay işleme + güncelleme + çizim hepsi bu pencerede biter
- Bütçeyi aşan tek bir fonksiyon kareyi düşürür

Tipik Bütçe Dağılımı

Olaylar: ~ 1 ms

Güncelleme: ~ 5 ms

Çizim: ~ 8 ms

Yedek: ~ 2 ms

Kural

Önce *ölç*, sonra *düzeltil*. Tahmin edilen darboğaz çoğu zaman gerçek darboğaz değildir.

cProfile ile Ölçüm

```
import cProfile, pstats

def main():
    # Oyun dongusu burada calisir
    ...

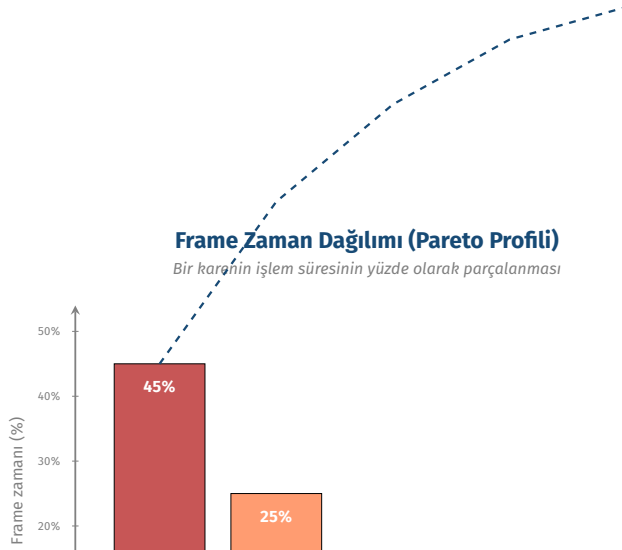
profiler = cProfile.Profile()
profiler.enable()
main()
profiler.disable()

stat = pstats.Stats(profiler).sort_stats("cumulative")
stat.print_stats(15)    # En yuksek 15 fonksiyon
```

Hedef

totttime ve cumtime sütunlarına bak: en üstteki birkaç fonksiyon %80 zamanı yer.

Pareto: Darboğazları Bulmak



Culling: Ekran Dışını Atla

```
viewport = pygame.Rect(kamera.x, kamera.y, GENISLIK, YUKSEKLİK)

for sprite in tum_spritelar:
    if not viewport.collidect(sprite.rect):
        continue                # Ne guncelle ne ciz
    sprite.guncelle(dt)
    ekran.blit(sprite.image, sprite.rect)
```

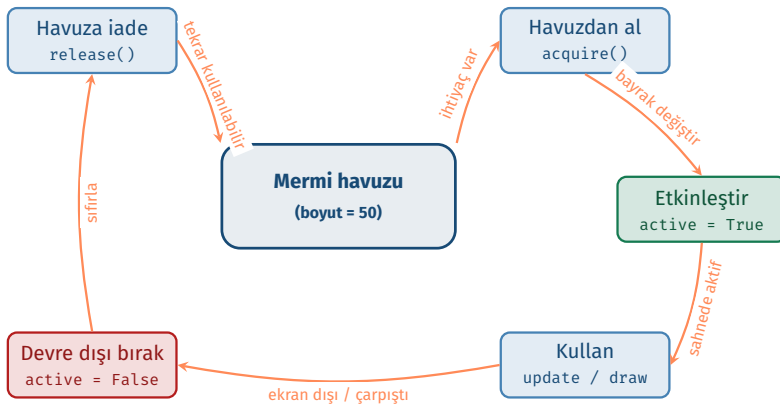
Mantık

Görünmeyen düşmanı çizmenin de güncellemenin de oyuncuya faydası yok. `collidect` $O(1)$ kontrol; ana çalışmadan kazanç çok büyük.

Dikkat

Görünmediği için ölmeyen, hareket etmeyen düşman *istemediğin* bir davranış olabilir. Mermi/parçacık güvenli; AI için viewport + tampon kullan.

Object Pool Yaşam Döngüsü



ObjectPool Sınıfı

```
class ObjectPool:
    def __init__(self, fabrika, kapasite=200):
        self.havuz = [fabrika() for _ in range(kapasite)]
        for nesne in self.havuz:
            nesne.aktif = False

    def al(self):
        for n in self.havuz:
            if not n.aktif:
                n.aktif = True
                return n
        return None          # Havuz dolu

    def birak(self, n):
        n.aktif = False      # Bellek serbest degil --- yeniden
                             # kullanim
```

Senaryo	Önce (FPS)	Sonra (FPS)
500 mermi (havuzsuz)	28	—
500 mermi (havuzlu)	—	60
2000 sprite (culling yok)	18	—
2000 sprite (culling var)	—	58
collide_rect ($O(n^2)$)	24	—
Spatial hash	—	60

Sonuç

Üç teknik birleştiğinde 60 FPS hedefi 18 FPS'lik bir başlangıçtan dahi tutturulabilir.

Gelişmiş Çarpışma

$O(n^2)$ Problemi

- Brute force: her sprite, her sprite ile karşılaştırılır
- 100 nesne $\rightarrow$ 4 950 kontrol
- 1 000 nesne $\rightarrow$ 499 500 kontrol
- 10 000 nesne $\rightarrow \sim$ 50 milyon kontrol

Sonuç

Çarpışma sayısı *karesel* büyür; 60 FPS bütçesi 1 000 nesnede çoktan aşılır.

Çözüm

Önce *geniş faz*: ucuz bir filtre olası çiftleri çıkarır. Sonra *dar faz*: pahalı kontrol sadece bu çiftler için yapılır.

Geniş Faz vs. Dar Faz

Geniş Faz (Broad-Phase)

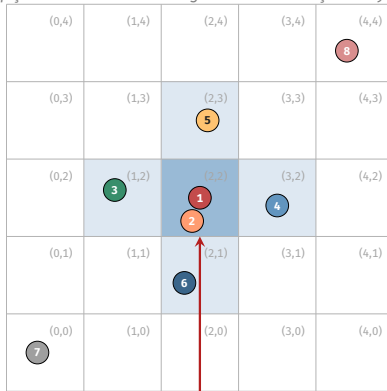
- Hızlı, kaba filtreleme: “hangi çiftler *olabilir*?”
- AABB (axis-aligned bounding box) kontrolü; `Rect.collidect`
- Veri yapısı: spatial hash, quad tree, sweep-and-prune

Dar Faz (Narrow-Phase)

- Pahalı, kesin kontrol: “çift *gerçekten* çarpışıyor mu?”
- Piksel maskesi, dairesel mesafe, çokgen kesişimi
- Sadece geniş fazdan geçen çiftler için çalıştırılır

Spatial Hash Grid (5×5)

Çarpışma denetimi sadece sorgu hücresi ve komşularda yapılır



- sorgu hücresi
- komşu (test edilen)

1 ile 8 arası: sahnedeki sprite'lar.

Yalnız aynı hücredeki veya 4 komşudaki nesneler test edilir: {1,2,5,6} ile 3,4

sorgu pozisyonu

Spatial Hash: Kod

```
class SpatialHash:
    def __init__(self, hucre=64):
        self.hucre = hucre
        self.tablo = {}          # (cx, cy) -> [sprite, ...]

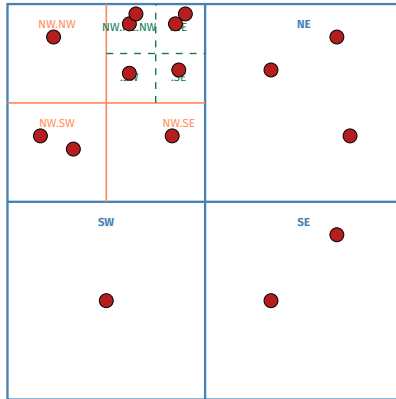
    def _hucreler(self, rect):
        x0, y0 = rect.left // self.hucre, rect.top // self.hucre
        x1, y1 = rect.right // self.hucre, rect.bottom // self.hucre
        return [(x, y) for x in range(x0, x1+1)
                for y in range(y0, y1+1)]

    def ekle(self, sprite):
        for h in self._hucreler(sprite.rect):
            self.tablo.setdefault(h, []).append(sprite)
```

Quad Tree: Uyarlanmış Bölme

Quad Tree Alt Bölme

Yoğunluk arttıkça hücre kendini dörde böler



Kural:

Her hücrede en fazla
capacity = 4 nesne.
Aşılırsa dörde bölünür.

Renk anlamı:

mavi: 1. seviye

turuncu: 2. seviye

yeşil: 3. seviye

Avantaj:

Boş alanlar bölünmez,
yoğun alanlar daha
detaylı sorgulanır.

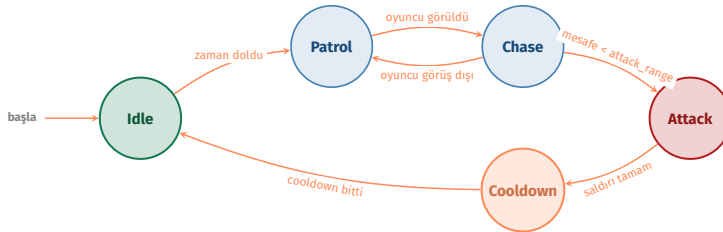
Sprite	Brute Force	Spatial Hash	Quad Tree
100	0.3 ms	0.4 ms	0.6 ms
500	7.5 ms	1.2 ms	1.8 ms
1 000	30 ms	2.4 ms	3.5 ms
5 000	750 ms	12 ms	18 ms

Pratik Tavsiye

- < 100 nesne: brute force yeter
- Düzgün dağılım, sabit yoğunluk: **spatial hash**
- Değişken yoğunluk (boş alan + kalabalık küme): **quad tree**

Temel AI Teknikleri

Düşman Sonlu Durum Makinesi (FSM)



Idle: bekler, çevreye bakar **Patrol:** iki nokta arasında devriye gezer
Chase: oyuncuyu kovalar **Attack:** menzilde ise saldırır
Cooldown: saldırı sonrası dinlenir, kısa süre savunmasız

Durumlar

Idle → Patrol → Chase → Attack → Flee; her geçiş bir *koşul* ile tetiklenir (mesafe, can, görüş hattı).

FSM Çekirdek Kod

```
class Dusman:
    def __init__(self):
        self.durum = "patrol"

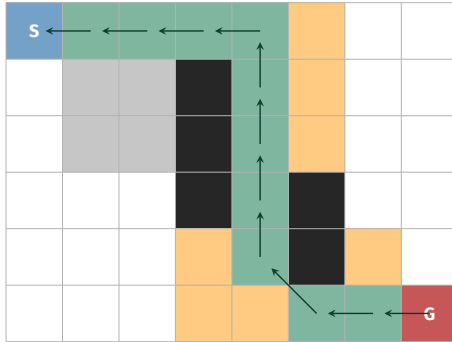
    def guncelle(self, oyuncu, dt):
        mesafe = self.pos.distance_to(oyuncu.pos)

        if self.durum == "patrol" and mesafe < 200:
            self.durum = "chase"
        elif self.durum == "chase" and mesafe < 40:
            self.durum = "attack"
        elif self.can < 20:
            self.durum = "flee"

        getattr(self, f"davran_{self.durum}")(oyuncu, dt)
```

A* Algoritması: Açık ve Kapalı Listeler

$S = \text{Start}, G = \text{Goal}$



Renk anlamı

Başlangıç (S)

Bitiş (G)

Engel

Açık liste

Kapalı liste

Bulunan yol

$$f = g + h$$

g: başlangıçtan

bu hücreye maliyet

h: hücreden hedefe

heuristik (ör. Manhattan)

Oklar: parent

yönünü gösterir.

Formül

$$f(n) = g(n) + h(n) \quad g: \text{başlangıçtan gelmenin maliyeti} \quad h: \text{hedefe tahmini mesafe (Manhattan)}$$

```
import heapq

def astar(baslangic, hedef, komsular, h):
    acik = [(0, baslangic)]
    geldi = {baslangic: None}
    g = {baslangic: 0}

    while acik:
        _, mevcut = heapq.heappop(acik)
        if mevcut == hedef:
            return yol_olustur(geldi, mevcut)
        for komsu in komsular(mevcut):
            yeni_g = g[mevcut] + 1
            if yeni_g < g.get(komsu, 1e9):
                g[komsu] = yeni_g
                f = yeni_g + h(komsu, hedef)
                heapq.heappush(acik, (f, komsu))
```

Steering Behaviors

Seek

Hedefe doğru sabit hızla.

$$\vec{v}_{\text{istek}} = (\widehat{\vec{p}_{\text{hedef}} - \vec{p}}) \cdot v_{\text{max}}$$

Arrival

Yaklaştıkça yavaşla; hedefte dur.

Flee

Seek'in tersi: hedeften kaçma.

Wander

Önündeki sanal daire üzerinde rastgele yön; doğal görünür.

Toplam Kuvvet

$$\vec{f} = w_1 \vec{f}_{\text{seek}} + w_2 \vec{f}_{\text{flee}} + w_3 \vec{f}_{\text{wander}} \quad (\text{vektör toplamı})$$

```
class GelismisDusman:
    def guncelle(self, oyuncu, harita, dt):
        # 1) FSM hangi davranis?
        if self.gorus_hatti(oyuncu):
            self.durum = "chase"

        # 2) A* hedefe yolu hesapla (her N karede bir)
        if self.durum == "chase" and self.yol_zamanlayici <= 0:
            self.yol = astar(self.tile, oyuncu.tile, harita.komsular
                              , manhattan)
            self.yol_zamanlayici = 0.5

        # 3) Steering yumusak hareket
        if self.yol:
            sonraki = self.yol[0]
            self.hiz = arrival(self.pos, sonraki.merkez, self.hiz)
        self.pos += self.hiz * dt
```


Üç Katman Birlikte

Katman 1: FSM

Düşman *ne* yapacağına karar verir (patrol / chase / flee).

Katman 2: A*

Düşman hedefe *nasıl* ulaşacağını planlar (tile bazlı yol).

Katman 3: Steering

Plan piksel bazında *yumuşak* hareket olur (varma, kaçma, dolaşma).

Bu Üçü Olmadan

Sadece FSM: katı, mekanik. Sadece A*: çocuk arabası. Sadece steering: hedefsiz dolaşma.
Birlikte: *akıllı görünen* düşman.

Oyun Dağıtımı

Niçin Dağıtım?

- Oyunculara python kurulu değil
- `pip install pygame` ister bilgi, ister yetki
- Çift tıklayıp oynayabilmeli
- Tek dosya dağıtmak ister

Çözüm

PyInstaller

Python yorumlayıcı + tüm
bağımlılıklar tek pakette

Sürpriz

“Bende çalışıyor” yetmez. Hedef makine *senin makinen değildir*, varsayımları test et.

PyInstaller Kullanımı

```
# Tek dosya, konsolsuz pencere
pyinstaller --onefile --windowed \
            --name "ZiplayanMacera" \
            --icon oyun_simge.ico \
            --add-data "assets:assets" \
            ana.py

# Sonuc:
# build/    --> ara dosyalar (silinebilir)
# dist/     --> ZiplayanMacera.exe (dagitilacak)
# *.spec    --> tekrar derleme tarifi
```

Bayraklar

-onefile: tek .exe. -windowed: konsol açılmasın. -add-data: assets klasörünü göm.

resource_path: Yol Sorunu

```
import sys, os

def resource_path(goreceli):
    """PyInstaller paketinde de gelistirme modunda da calisir."""
    if hasattr(sys, "_MEIPASS"):
        # Paketlenmis modda: gecici dizin
        taban = sys._MEIPASS
    else:
        # Normal modda: skript dizini
        taban = os.path.abspath(".")
    return os.path.join(taban, goreceli)

# Kullanım:
oyuncu_img = pygame.image.load(resource_path("assets/oyuncu.png"))
```

En Yaygın Hata

Doğrudan "assets/oyuncu.png" verirken: gelistirmede çalışır, paketlenmiş exe'de

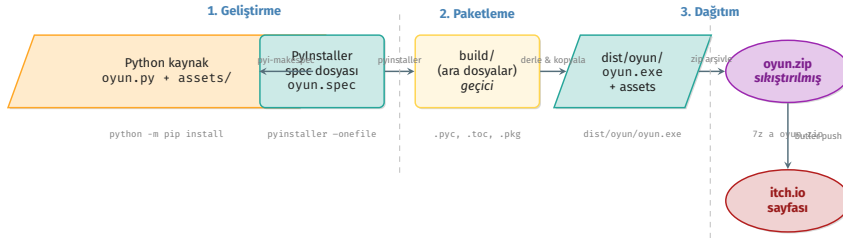
```
# ZiplayanMacera.spec
a = Analysis(
    ['ana.py'],
    pathex=[],
    datas=[('assets', 'assets'), ('save', 'save')],
    hiddenimports=['pygame._sdl2'],
)
pyz = PYZ(a.pure)
exe = EXE(pyz, a.scripts, a.binaries, a.datas,
          name='ZiplayanMacera',
          console=False,
          icon='oyun_simgesi.ico')
```

Tekrar Derleme

pyinstaller ZiplayanMacera.spec — bayrakları her seferinde yazmaktan kurtarır.

Dağıtım Pipeline: Python kaynaktan itch.io'ya

PyInstaller ile tek dosya/exe paketleme akışı



Not: spec dosyası bir kez üretilir; içine `datas=[(assets, assets)]` eklenip versiyonlanır. Sonraki derlemelerde `pyinstaller oyun.spec` yeterlidir.

Adımlar

Kod → PyInstaller → dist/ → ZIP → itch.io sayfası → butler push → oyuncu indirir.

Hazırlanacaklar

- Kapak görseli (630×500 px) ve ekran görüntüleri (en az 3)
- Açıklama: ne yapan oyun, kontroller, hikaye
- Türler/etiketler: *platformer*, *2D*, *pygame*
- Fiyat: ücretsiz, isteğe bağlı bağış, sabit ücret
- Platform: Windows / macOS / Linux ZIP'leri ayrı ayrı

butler ile Otomasyon

```
butler push dist/ZiplayanMacera.zip kullanıcı/oyun:windows
```


- ❶ **Performans:** cProfile ile ölç, culling + object pooling ile 60 FPS'ı tuttur
- ❷ **Çarpışma:** $O(n^2)$ 'den geniş faz / dar faz ayrımıyla çık; spatial hash veya quad tree seç
- ❸ **AI:** FSM (karar) + A* (yol) + steering (yumuşak hareket) üç katmanlı düşman
- ❹ **Dağıtım:** PyInstaller -onefile -windowed, resource_path, itch.io yayını

Bölüm Çıktısı

Optimize edilmiş Zıplayan Macera v5: 60 FPS, gelişmiş düşman AI'sı, çift tıklayıp oynanabilir .exe ve canlı bir itch.io sayfası.

Hafta 14: Dönem Projesi ve Sunum

- Dönem boyunca üretilen oyunun final cılası
- Demo videosu ve devkit hazırlığı
- Sınıfta sunum, akran değerlendirmesi
- Portföy için itch.io profili

Sorular?